

5 BACKEND PROJECTS EVERY GOAT HAS BUILT

Unrecognizable from the usual to-do app and blog API list — ranked difficulty 1 to 5.

DIFFICULTY LADDER

01	● ○ ○ ○ ○	Token Bucket Rate Limiter Service	BEGINNER
02	● ● ○ ○ ○	Reliable Job Queue with Dead-Letter Recovery	EASY-MED
03	● ● ● ○ ○	Event-Sourced Ledger (Double-Entry Bank Core)	INTERMEDIATE
04	● ● ● ● ○	Real-Time Collaborative Sync Engine	ADVANCED
05	● ● ● ● ●	Self-Healing Distributed Cache	EXPERT

HOW TO USE THIS

Every requirement below is described by behavior, not by tooling — build it in whatever language or stack you already know. Treat each page as a spec, not a tutorial: no boilerplate, no starter repo. The constraint is the lesson.

PROJECT 01 ●○○○○

BEGINNER

Token Bucket Rate Limiter Service

A standalone API that rate-limits other APIs — not a feature, a product.

WHY THIS ISN'T A GENERIC PROJECT

Every backend eventually needs rate limiting, but almost nobody builds it as its own networked service that other services call into. This forces you to think about shared state, clock precision, and concurrency instead of just importing a library.

CORE REQUIREMENTS

- 01 Expose an endpoint that, given a client key, returns ALLOW or DENY based on a token bucket algorithm.
- 02 Support per-client configurable limits (requests per second, burst size) set via an admin endpoint.
- 03 State must survive a service restart — persist bucket state, don't keep it only in memory.
- 04 Handle concurrent requests for the SAME client key without letting tokens be double-spent (race condition safe).
- 05 Provide a sliding-window mode in addition to token bucket, selectable per client.
- 06 Return standard rate-limit headers (limit, remaining, reset time) on every response.
- 07 Include a load test proving correctness under 500+ concurrent requests per second.

STRETCH GOALS

- + Add a distributed mode where multiple instances of your limiter share state correctly.
- + Build a tiny dashboard showing live request/deny rates per client.

SKILLS YOU'LL MASTER

Concurrency control, atomic operations, algorithm design, API contracts, load testing.

PROJECT 02 ●●○○○

EASY-MED

Reliable Job Queue with Dead-Letter Recovery

Build your own background-job engine — the thing every framework hides from you.

WHY THIS ISN'T A GENERIC PROJECT

Using Celery or Sidekiq teaches you the API, not the mechanics. Building the queue yourself forces you to confront exactly-once delivery myths, retry storms, and what 'reliable' really means when machines crash mid-task.

CORE REQUIREMENTS

- 01 Workers pull jobs from a durable queue (not an in-memory list) — jobs must survive a crash.
- 02 Support job priorities and scheduled (delayed) execution, not just immediate dispatch.
- 03 Implement automatic retries with exponential backoff on failure, capped at a max attempt count.
- 04 Jobs that exceed max retries move to a dead-letter queue that can be inspected and replayed.
- 05 Guarantee a job is never processed by two workers at once, even if a worker dies mid-job (visibility timeout / lease pattern).
- 06 Expose an endpoint or CLI to inspect queue depth, in-flight jobs, and dead-letter contents.
- 07 Demonstrate idempotency: re-running a job that partially completed does not double-apply side effects.

STRETCH GOALS

- + Add job dependencies (job B only runs after job A succeeds).
- + Build a simple web UI to visualize queue health in real time.

SKILLS YOU'LL MASTER

Distributed locking, fault tolerance, idempotency design, queueing theory, observability.

PROJECT 03 ●●●○○

INTERMEDIATE

Event-Sourced Ledger (Double-Entry Bank Core)

Model money the way real banks do: as an immutable stream of events, never mutable balances.

WHY THIS ISN'T A GENERIC PROJECT

A CRUD 'accounts' table with a balance column is how amateurs model money and how bugs happen. This project forces you to learn event sourcing, double-entry accounting invariants, and how to rebuild state from history — skills that transfer directly to fintech, audit systems, and anything where 'how did we get here' matters.

CORE REQUIREMENTS

- 01 Every balance change is recorded as an immutable, append-only event — never UPDATE a balance directly.
- 02 Enforce true double-entry bookkeeping: every transaction has matching debit and credit entries that sum to zero.
- 03 Current account balance is always a DERIVED value computed (or replayed) from its event history.
- 04 Support point-in-time queries: 'what was this account's balance as of last Tuesday?'
- 05 Reject any transaction that would violate invariants (e.g. overdraft on a non-credit account) atomically — no partial transfers.
- 06 Provide a full audit trail endpoint that reconstructs and explains how an account reached its current balance.
- 07 Handle concurrent transfers between the same two accounts without corrupting balances (no lost updates).

STRETCH GOALS

- + Add snapshotting so balance recomputation doesn't replay the entire event history every time.
- + Support multi-currency ledgers with exchange-rate-aware transfers.

SKILLS YOU'LL MASTER

Event sourcing, transactional integrity, domain modeling, concurrency, financial correctness.

PROJECT 04 ●●●●○

ADVANCED

Real-Time Collaborative Sync Engine

The backend behind Google Docs-style live editing — built from first principles.

WHY THIS ISN'T A GENERIC PROJECT

Most developers consume sync engines (Firebase, Liveblocks) without understanding how concurrent edits are merged without conflict. Implementing the conflict-resolution logic yourself is one of the most respected, least-attempted backend skills there is.

CORE REQUIREMENTS

- 01** Multiple clients can edit the SAME shared document concurrently and converge to an identical final state.
- 02** Implement a conflict resolution strategy yourself — either Operational Transformation or a CRDT (no pulling in a pre-built CRDT library).
- 03** Edits made offline must merge correctly once a client reconnects, without overwriting others' changes.
- 04** Maintain full edit history and support reverting the document to any earlier point in time.
- 05** Broadcast live changes to all connected clients with sub-second propagation.
- 06** Prove convergence formally: write a test that applies the same set of edits in different orders across simulated clients and asserts identical final state.
- 07** Handle a client disconnect/reconnect mid-edit without data loss or duplication.

STRETCH GOALS

- + Extend beyond plain text to a structured document (e.g. nested lists or a simple spreadsheet grid).
- + Add presence indicators (who's online, where their cursor is) as a secondary real-time channel.

SKILLS YOU'LL MASTER

CRDTs/OT, real-time protocols, distributed consistency, eventual convergence proofs.

PROJECT 05 ●●●●●

EXPERT

Self-Healing Distributed Cache

Build your own Redis cluster: sharded, replicated, and able to survive node failure live.

WHY THIS ISN'T A GENERIC PROJECT

Anyone can call SET and GET on Redis. Almost nobody has built the thing underneath: consistent hashing, replica failover, and rebalancing without downtime. This is the project that proves you understand distributed systems, not just distributed tools.

CORE REQUIREMENTS

- 01** Run as multiple independent cache nodes that together form one logical cache cluster.
- 02** Use consistent hashing (not modulo hashing) to distribute keys, so adding/removing a node reshuffles a minimal number of keys.
- 03** Replicate every key to at least one additional node; reads must still succeed if the primary holder of a key goes down.
- 04** Detect a failed node automatically (heartbeat/gossip) and reroute its traffic without manual intervention.
- 05** When a failed node returns (or a new node joins), automatically rebalance keys with zero downtime for clients.
- 06** Support TTL-based expiry that is honored consistently across all replicas of a key.
- 07** Demonstrate the cluster surviving a live kill of a random node mid-traffic with zero failed client requests (beyond one configurable retry).

STRETCH GOALS

- + Add an LRU/LFU eviction policy under memory pressure, per node.
- + Build a cluster-wide consistency mode (quorum reads/writes) as an optional stricter alternative to eventual consistency.

SKILLS YOU'LL MASTER

Consistent hashing, gossip/failure detection, replication, rebalancing, CAP trade-offs.